

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Τομέας Τεχνολογίας, Πληροφορικής και Υπολογιστών
Εργαστήριο Υπολογιστικών Συστημάτων

Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών
1^η Εργαστηριακή Άσκηση



Στοιχεία Φοιτητή

- Όνομα: Πέππας Μιχαήλ-Αθανάσιος
- Αριθμός Μητρώου: 03121026
- Email: el21026@mail.ntua.gr
- Ημερομηνία Παράδοσης Αναφοράς: 31.04.2026

Περιεχόμενα

1	Μεθοδολογία, μετρικές και έλεγχος δεδομένων	3
2	5.2 Ανάλυση εντολών άλματος	4
2.1	Σκοπός και μεθοδολογία	4
3	5.3 Μηχανισμοί n-bit πρόβλεψης και εναλλακτικά FSMs	6
3.1	Θεωρητικό υπόβαθρο	6
3.2	5.3(i): Σταθερές 16K BHT entries	6
3.3	5.3(ii): Εναλλακτικά 2-bit FSMs του Nair	7
3.4	5.3(iii): Σταθερό hardware budget 32K bits	8
4	5.4 Μελέτη του BTB	9
4.1	Υλοποίηση και μετρικές	9
4.2	Αποτελέσματα και επιλογή	10
5	5.5 Μελέτη του RAS	12
5.1	Θεωρία και υλοποίηση	12
5.2	Αποτελέσματα και επιλογή	12
6	5.6.1 Perceptrons	14
6.1	Θεωρητικό υπόβαθρο	14
6.2	Σχολιασμός αποτελεσμάτων	15
7	5.6.2 Σύγκριση διαφορετικών μηχανισμών πρόβλεψης	16
7.1	Υλοποίηση και επιλογές hardware	16
7.2	Αποτελέσματα και τελική επιλογή	19
8	5.7 Ακρίβεια προσομοιώσεων με ref inputs	20
8.1	Επιλογή μηχανισμών πρόβλεψης για έλεγχο	20
8.2	Σύγκριση συμπερασμάτων	21
9	Συνολικά συμπεράσματα	22

1 Μεθοδολογία, μετρικές και έλεγχος δεδομένων

Στην άσκηση μελετήσαμε μηχανισμούς πρόβλεψης εντολών άλματος μέσω του dynamic binary instrumentation εργαλείου PIN. Ο βοηθητικός κώδικας παρέχει δύο βασικά pintools: το `cslab_branch_stats.cpp`, το οποίο συλλέγει στατιστικά για τις εντολές άλματος, και το `cslab_branch.cpp`, το οποίο προσομοιώνει μηχανισμούς πρόβλεψης, BTB και RAS. Στο δεύτερο επεκτείναμε τις διαθέσιμες επιλογές ώστε να καλύπτονται ακριβώς οι διαμορφώσεις που ζητούνται στα ερωτήματα 5.3–5.7.

Η εκφώνηση ορίζει ότι τα ερωτήματα 5.3–5.6 εκτελούνται με τα `train inputs`, ενώ το 5.2 ζητά και `train` και `ref`. Το 5.7 αποτελεί έλεγχο εγκυρότητας σε `ref inputs` για τρεις μηχανισμούς πρόβλεψης που επιλέγονται μετά την ολοκλήρωση της σύγκρισης του 5.6.2. Τα αποτελέσματα αποθηκεύτηκαν ανά ενότητα σε raw αρχεία, CSV summaries και συνοπτικά text summaries, ώστε οι πίνακες και τα διαγράμματα της αναφοράς να προκύπτουν αναπαραγωγή από τα ίδια δεδομένα.

Η κύρια μετρική για τους μηχανισμούς direction prediction είναι:

$$\text{direction MPKI} = \frac{\text{direction mispredictions}}{\text{total instructions}} \cdot 1000.$$

Χρησιμοποιήσαμε δύο συμπληρωματικούς μέσους όρους. Ο αριθμητικός μέσος του MPKI δίνει ίσο βάρος σε κάθε benchmark και χρησιμοποιείται ως κύριο κριτήριο επιλογής, επειδή δεν αφήνει τα πολύ μεγάλα benchmarks να κυριαρχήσουν πλήρως στο συμπέρασμα. Το aggregate MPKI υπολογίζεται ως:

$$\text{aggregate MPKI} = \frac{\sum \text{mispredictions}}{\sum \text{instructions}} \cdot 1000,$$

και δείχνει την επίδοση όταν κάθε δυναμική εντολή έχει ίσο βάρος. Δεν χρησιμοποιήσαμε γεωμετρικό μέσο για το MPKI, επειδή δεν πρόκειται για λόγο επιτάχυνσης αλλά για φυσική μετρική αστοχιών ανά χίλιες εντολές. Η διάκριση αυτή είναι χρήσιμη θεωρητικά: το MPKI είναι κανονικοποιημένο πλήθος αστοχιών, άρα ο αριθμητικός και ο instruction-weighted μέσος έχουν άμεση ερμηνεία, ενώ ένας γεωμετρικός μέσος θα έκρυβε το πραγματικό πλήθος των λαθών.

Πριν γραφτεί η αναφορά, ελέγχθηκαν όλα τα benchmark outputs. Δεν υπήρχαν missing ή unparseable raw outputs: 22 outputs για το 5.2, 11 outputs για καθεμία από τις ενότητες 5.3, 5.4, 5.5, 5.6.1, 5.6.2 και 11 ref outputs για το 5.7. Τα μόνα μη κενά stderr logs εμφανίζονται στο 416.gamess και αντιστοιχούν σε γνωστές αριθμητικές προειδοποιήσεις ή

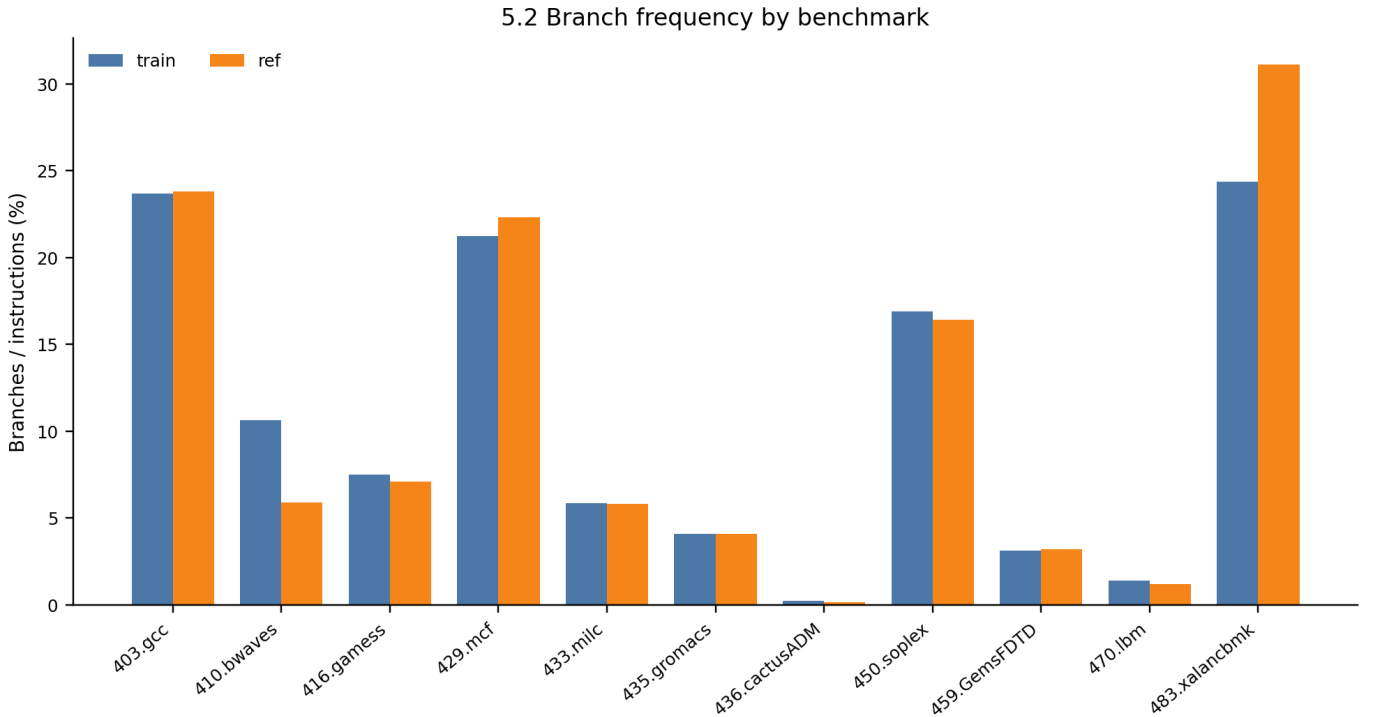
τερματισμό του benchmark, όχι σε αποτυχία του pintool, αφού τα αντίστοιχα raw outputs έγιναν parse κανονικά.

2 5.2 Ανάλυση εντολών άλματος

2.1 Σκοπός και μεθοδολογία

Το πρώτο πειραματικό βήμα δεν αξιολογεί ακόμη μηχανισμούς πρόβλεψης. Στόχος του είναι να καταγραφεί το δυναμικό μίγμα των branches για κάθε benchmark και για τα δύο input sets. Μετρήσαμε το πλήθος των συνολικών εντολών, το πλήθος των branches και την κατανομή τους σε conditional taken, conditional not-taken, unconditional, calls και returns. Το branch frequency υπολογίζεται ως:

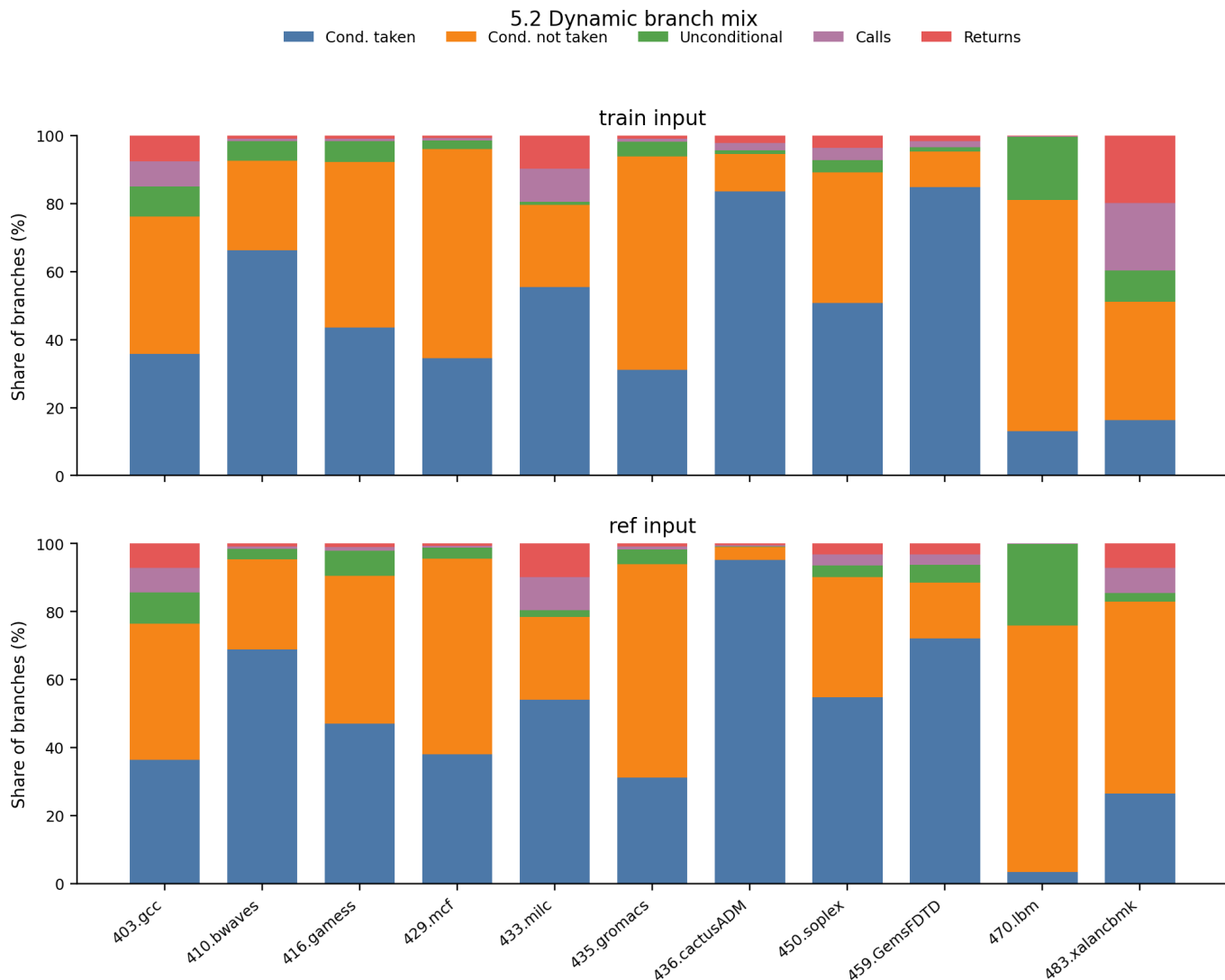
$$\text{branch frequency} = \frac{\text{total branches}}{\text{total instructions}} \cdot 100\%.$$



Σχήμα 1: Συχνότητα εντολών άλματος για train και ref inputs.

Από το Σχήμα 1 προκύπτει ότι τα benchmarks δεν επιβαρύνουν στον ίδιο βαθμό τον μηχανισμό πρόβλεψης. Στο train input τα πιο branch-heavy benchmarks είναι τα 483.xalancbmk με 24.37%, 403.gcc με 23.69%, 429.mcf με 21.22% και 450.soplex με 16.89%. Αντίθετα, τα 436.cactusADM και 470.lbm έχουν πολύ μικρό branch frequency, 0.21% και 1.41% αντίστοιχα. Η ίδια γενική εικόνα διατηρείται και στο ref, με το 483.xalancbmk να αυξάνει ακόμη περισσότερο το branch frequency στο 31.10%.

Η παρατήρηση αυτή είναι σημαντική για την ερμηνεία του MPKI. Ένας μηχανισμός πρόβλεψης μπορεί να έχει παρόμοιο ποσοστό αστοχίας σε δύο benchmarks, αλλά να οδηγεί σε πολύ διαφορετικό MPKI αν το ένα benchmark εκτελεί πολύ περισσότερα branches ανά χίλιες εντολές. Με άλλα λόγια, το branch frequency λειτουργεί ως σύνδεσμος ανάμεσα στη μικροαρχιτεκτονική ακρίβεια του predictor και στον πραγματικό φόρτο που δέχεται το frontend του επεξεργαστή.



Σχήμα 2: Μίγμα κατηγοριών branches ως ποσοστό των συνολικών branches.

Το Σχήμα 2 δείχνει ότι τα conditional branches κυριαρχούν στα περισσότερα benchmarks, αλλά υπάρχουν ουσιαστικές διαφοροποιήσεις. Για παράδειγμα, στο 483.xalancbmk το **train input** έχει μεγάλο ποσοστό calls και returns, περίπου 19.82% το καθένα, επομένως αναμένεται έντονη επίδραση από τη RAS. Αντίθετα, το 470.lbm έχει χαμηλό branch frequency αλλά σχετικά μεγάλο ποσοστό unconditional branches, 18.61% στο **train** και

24.05% στο ref. Επομένως, η μετέπειτα αξιολόγηση δεν πρέπει να περιοριστεί μόνο στο direction prediction, αλλά να ξεχωρίσει direction prediction, target prediction μέσω BTB και πρόβλεψη επιστροφών μέσω RAS.

3 5.3 Μηχανισμοί n-bit πρόβλεψης και εναλλακτικά FSMs

3.1 Θεωρητικό υπόβαθρο

Οι μηχανισμοί n-bit πρόβλεψης βασίζονται σε έναν Branch History Table (BHT). Κάθε στατική εντολή άλματος προσπελαίνει μία εγγραφή του BHT μέσω bits του PC και η εγγραφή περιέχει έναν saturating up-down counter. Σε κάθε taken outcome ο counter αυξάνεται μέχρι τη μέγιστη τιμή του, ενώ σε κάθε not-taken outcome μειώνεται μέχρι το μηδέν. Η πρόβλεψη προκύπτει από το πιο σημαντικό bit του counter. Επειδή ο BHT είναι πεπερασμένος, διαφορετικά branches μπορούν να χαρτογραφηθούν στην ίδια εγγραφή και να δημιουργήσουν aliasing.

Ο κλασικός 2-bit μηχανισμός είναι ιδιαίτερα χρήσιμος για loops, επειδή έχει hysteresis. Για μια τυπική ακολουθία loop branch:

$$T, T, T, \dots, T, N,$$

η μία not-taken έξοδος του loop δεν αρκεί για να αλλάξει αμέσως μια ισχυρή taken πρόβλεψη. Έτσι αποφεύγεται η διπλή αστοχία που συχνά εμφανίζεται με 1-bit μηχανισμούς. Το βασικό θεωρητικό trade-off είναι επομένως ανάμεσα σε περισσότερες entries, που μειώνουν το aliasing, και περισσότερα bits ανά entry, που αυξάνουν το hysteresis αλλά ενδέχεται να καθυστερούν την προσαρμογή σε πραγματική αλλαγή συμπεριφοράς.

3.2 5.3(i): Σταθερές 16K BHT entries

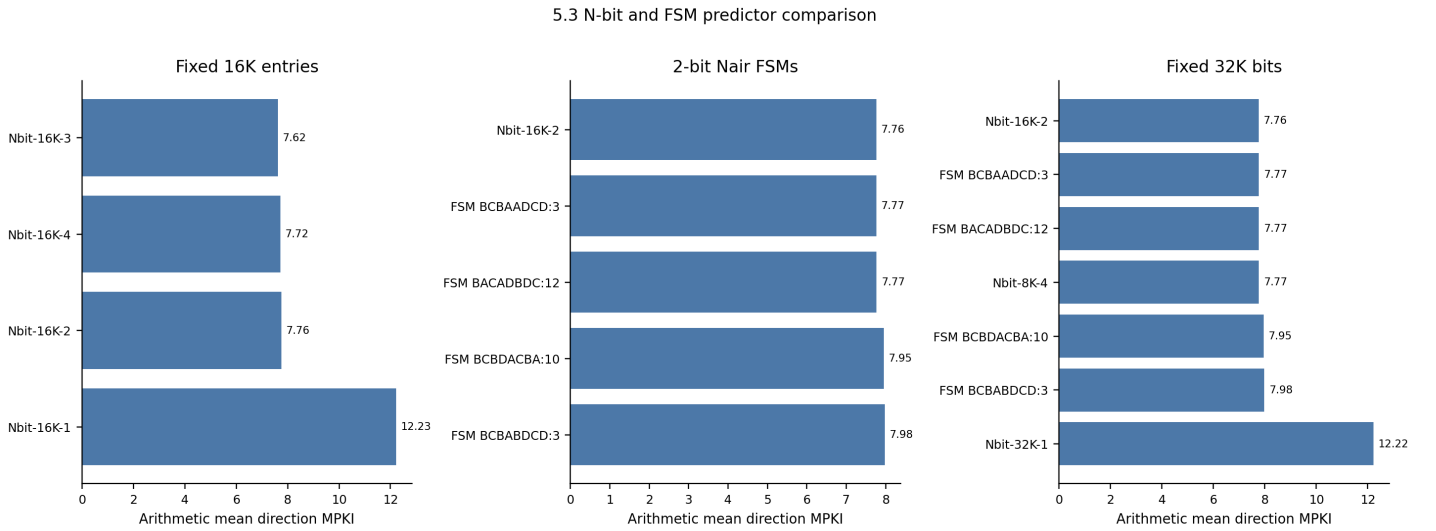
Στο πρώτο υποερώτημα κρατήσαμε σταθερό το πλήθος των BHT entries στις 16K και αλλάξαμε το πλήθος bits ανά entry. Τα αποτελέσματα συνοψίζονται στον Πίνακα 1.

Predictor	Bits	Mean MPKI	Aggregate MPKI
Nbit-16K-1	16384	12.229	10.516
Nbit-16K-2	32768	7.761	6.505
Nbit-16K-3	49152	7.622	6.459
Nbit-16K-4	65536	7.717	6.531

Με αυστηρή ανάγνωση του ερωτήματος, όπου το πλήθος entries είναι σταθερό και δεν επιβάλλεται budget, θα επέλεγα τον Nbit-16K-3, επειδή έχει τη μικρότερη αριθμητική μέση τιμή του MPKI. Η βελτίωσή του όμως ως προς τον 2-bit μηχανισμό είναι μικρή, περίπου 0.139 MPKI, ενώ απαιτεί 50% περισσότερα bits. Ο 4-bit μηχανισμός δεν βελτιώνει περαιτέρω την επίδοση, γεγονός που υποδηλώνει ότι το πρόσθετο hysteresis καθυστερεί την προσαρμογή χωρίς να μειώνει ουσιαστικά το aliasing.

3.3 5.3(ii): Εναλλακτικά 2-bit FSMs του Nair

Στο δεύτερο υποερώτημα υλοποιήσαμε τα εναλλακτικά 4-state FSMs του πίνακα VI του Nair. Η κωδικοποίηση ενός FSM, όπως BCBAADCD:3, περιγράφει τις μεταβάσεις των καταστάσεων A, B, C, D για not-taken και taken outcome, ενώ ο αριθμός μετά την άνω-κάτω τελεία ορίζει ποιες καταστάσεις προβλέπουν taken. Το ABACBD:3 είναι ο κλασικός saturating counter και εκπροσωπείται από τον Nbit-16K-2.



Σχήμα 3: Σύγκριση μηχανισμών n-bit πρόβλεψης και Nair FSMs με αριθμητικό μέσο του direction MPKI.

Τα καλύτερα FSMs είναι πρακτικά ισοδύναμα με τον κλασικό 2-bit μηχανισμό. Η διαμόρφωση Nbit-16K-2 έχει mean MPKI 7.761, η FSM-16K-BCBAADCD-3 έχει 7.767 και η FSM-16K-BACADBDC-12 έχει 7.770. Οι διαφορές είναι τόσο μικρές ώστε δεν δικαιολογούν την επιλογή πιο ειδικού FSM αντί του κλασικού saturating counter. Θεωρητικά, τα FSMs του Nair εξετάζουν διαφορετικές πολιτικές μετάβασης ανάμεσα στις ίδιες τέσσερις καταστάσεις και επομένως διαφορετικό τρόπο ενσωμάτωσης του hysteresis. Η μέτρηση δείχνει ότι η κλασική διάταξη δύο bits βρίσκεται ήδη πολύ κοντά στο βέλτιστο για τα συγκεκριμέ-

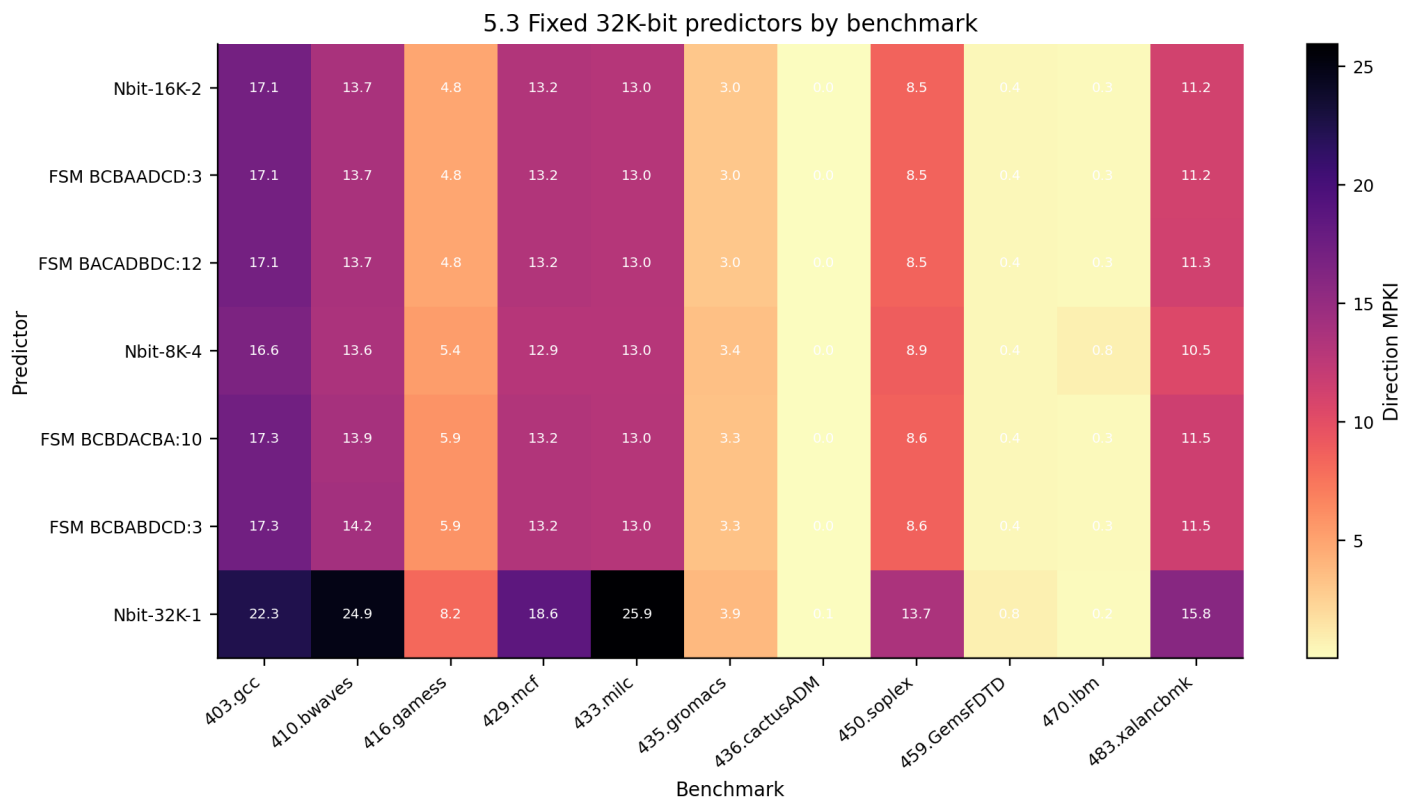
να traces. Επομένως, για το 5.3(ii) επιλέγουμε τη διαμόρφωση Nbit-16K-2, κυρίως λόγω απλότητας και ισοδύναμης επίδοσης.

3.4 5.3(iii): Σταθερό hardware budget 32K bits

Στο τρίτο υποερώτημα το κρίσιμο σημείο είναι ότι το hardware μένει σταθερό στα 32K bits. Συνεπώς, η αύξηση των bits ανά counter μειώνει αναγκαστικά το πλήθος των entries:

1-bit : 32K entries, 2-bit : 16K entries, 4-bit : 8K entries.

Με αυτόν τον τρόπο αξιολογείται άμεσα η ανταλλαγή ανάμεσα σε λιγότερο aliasing και περισσότερο hysteresis ανά entry.



Σχήμα 4: Τιμή direction MPKI ανά benchmark για τους 7 μηχανισμούς του πειράματος fixed-32K.

Predictor	Mean MPKI	Aggregate MPKI
Nbit-16K-2	7.761	6.505
FSM-16K-BCBAADCD-3	7.767	6.506
FSM-16K-BACADBDC-12	7.770	6.520
Nbit-8K-4	7.770	6.569
FSM-16K-BCBDACBA-10	7.954	6.807
FSM-16K-BCBABDCD-3	7.979	6.858
Nbit-32K-1	12.217	10.516

Πίνακας 2: Σύγκριση μηχανισμών πρόβλεψης με σταθερό κόστος 32K bits.

Ο 1-bit μηχανισμός, παρότι έχει τις περισσότερες entries, υστερεί έντονα επειδή δεν διαθέτει hysteresis. Ο 4-bit μηχανισμός έχει μεγαλύτερο hysteresis αλλά λιγότερες entries, άρα αυξάνει το aliasing. Ο καλύτερος συμβιβασμός εμφανίζεται στη διαμόρφωση Nbit-16K-2. Η συγκεκριμένη διαμόρφωση είναι πρώτη με βάση τον αριθμητικό μέσο του MPKI, σχεδόν πρώτη και ως προς το aggregate MPKI, και παραμένει ο πιο απλός 2-bit μηχανισμός. Τη χρησιμοποιούμε στη συνέχεια ως τον εκπρόσωπο των n-bit predictors στο 5.6.2.

4 5.4 Μελέτη του BTB

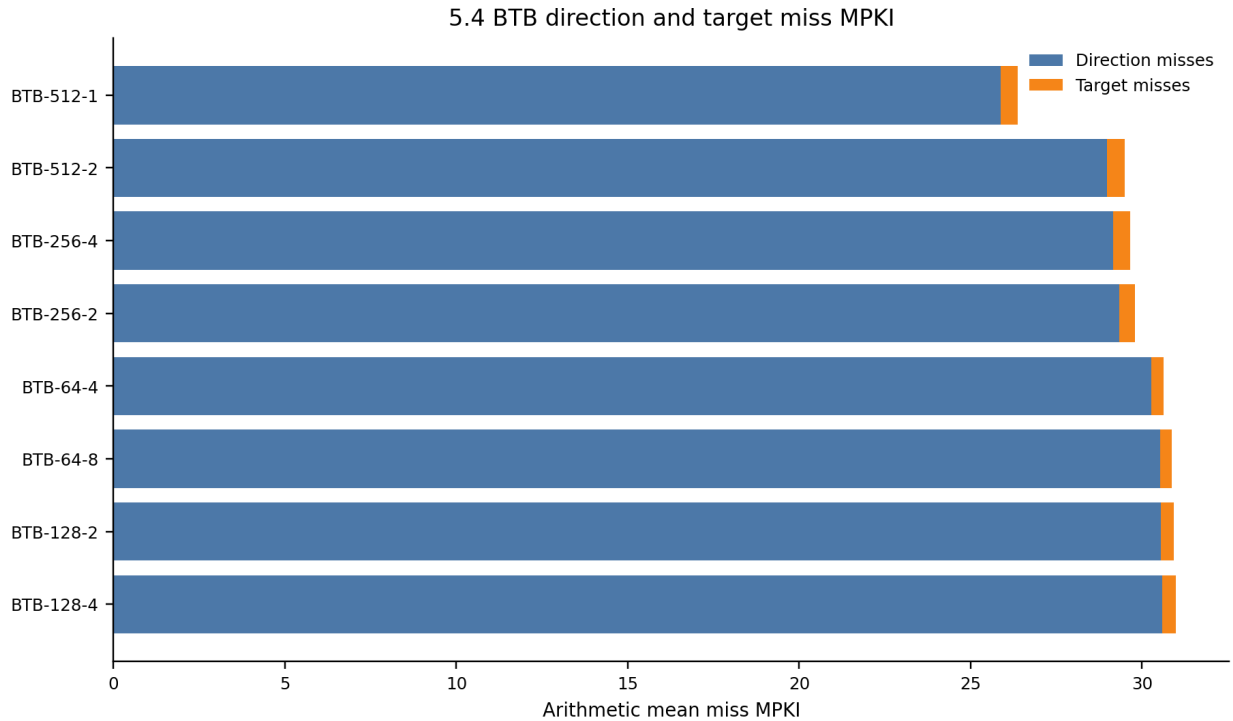
4.1 Υλοποίηση και μετρικές

Ο Branch Target Buffer (BTB) χρησιμοποιείται για να προβλέψει γρήγορα τόσο ότι ένα branch θα είναι taken όσο και ποιος θα είναι ο target PC. Τον υλοποιήσαμε ως cache-like δομή με valid bit, πλήρες PC ως tag, αποθηκευμένο target και LRU replacement ανά set. Για κάθε οργάνωση:

$$\text{sets} = \frac{\text{total entries}}{\text{associativity}}.$$

Ένα BTB hit προβλέπει taken και δίνει το αποθηκευμένο target. Ένα miss προβλέπει not-taken. Οι εγγραφές ενημερώνονται μόνο όταν το πραγματικό αποτέλεσμα είναι taken, επειδή μόνο τότε υπάρχει target που πρέπει να αποθηκευτεί. Η οργάνωση αυτή είναι άμεσα ανάλογη με μια cache: οι περισσότερες entries μειώνουν capacity misses, ενώ το μεγαλύτερο associativity μειώνει conflict misses αλλά αφήνει λιγότερα sets για το ίδιο συνολικό μέγεθος και αυξάνει το κόστος σύγκρισης tags.

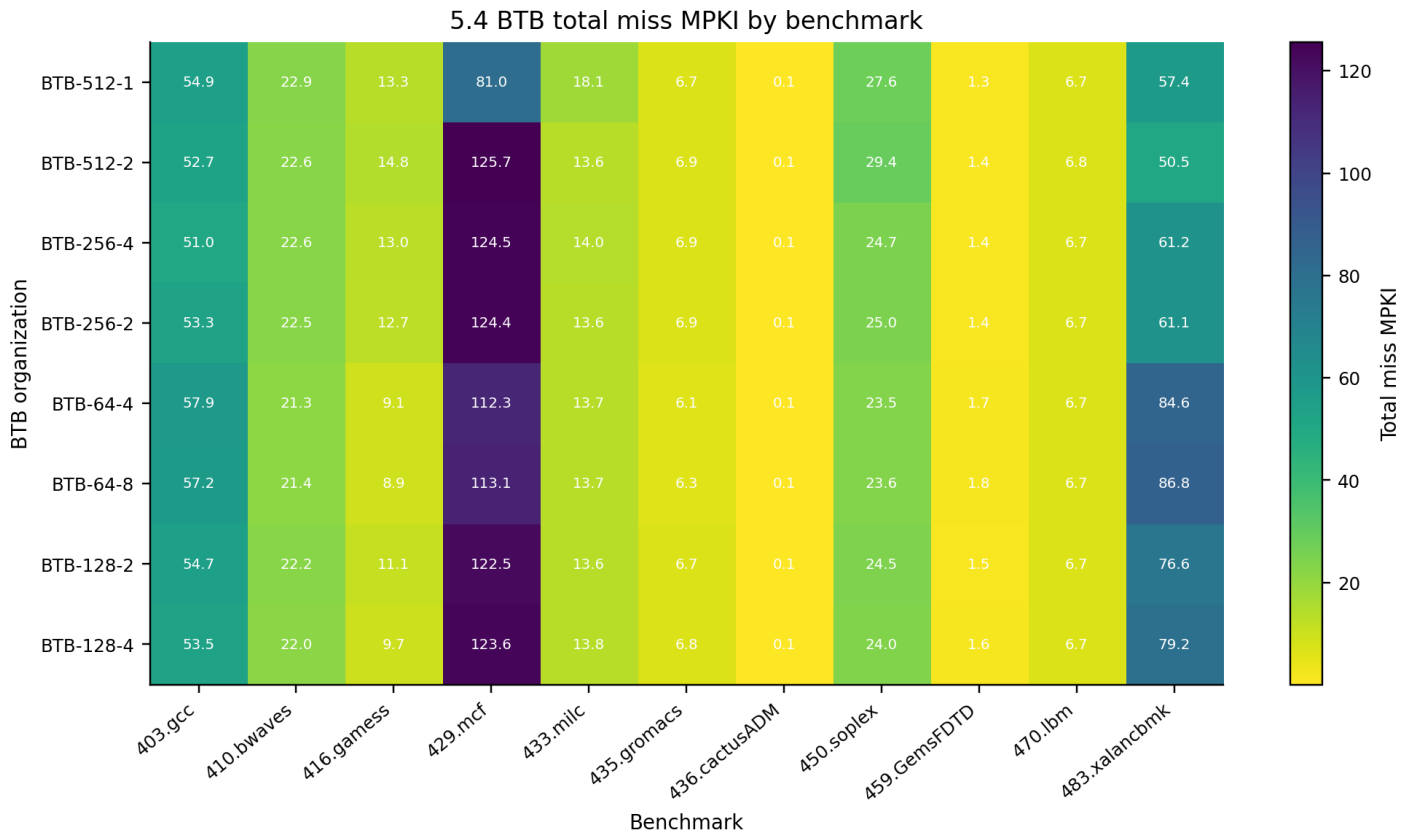
Υπάρχουν δύο είδη αστοχίας. Direction miss εμφανίζεται όταν η απόφαση taken/not-taken είναι λάθος. Target miss εμφανίζεται όταν το direction prediction έχει σωστά προβλέψει taken, αλλά το αποθηκευμένο target διαφέρει από το πραγματικό. Επομένως, τα target misses είναι πιο ειδική περίπτωση και σχετίζονται κυρίως με branches που αλλάζουν target δυναμικά.



Σχήμα 5: Τιμές direction και target miss MPKI για τις οργανώσεις BTB.

4.2 Αποτελέσματα και επιλογή

Τα target misses είναι πολύ μικρότερα από τα direction misses. Για παράδειγμα, στον BTB-512-1 ο αριθμητικός μέσος direction miss rate είναι 23.12%, ενώ ο αντίστοιχος target miss rate είναι μόλις 0.62%. Επομένως, στα συγκεκριμένα traces η επίδοση του BTB καθορίζεται κυρίως από την ικανότητά του να αναγνωρίζει εγκαίρως τα taken branches και δευτερευόντως από τη διάκριση μεταξύ διαφορετικών targets για ήδη σωστά προβλεπόμενα taken branches.



Σχήμα 6: Το total BTB miss MPKI ανά benchmark και οργάνωση.

BTB	Sets	Mean total MPKI	Aggregate total MPKI	Mean target miss%
BTB-512-1	512	26.370	18.926	0.621
BTB-512-2	256	29.499	18.510	0.571
BTB-256-4	64	29.646	19.838	0.568
BTB-256-2	128	29.798	19.766	0.594
BTB-64-4	16	30.629	22.167	0.635
BTB-64-8	8	30.868	22.570	0.603
BTB-128-2	64	30.925	21.700	0.570
BTB-128-4	32	30.990	21.926	0.590

Πίνακας 3: Συνοπτικά αποτελέσματα BTB.

Με κύριο κριτήριο τον αριθμητικό μέσο του total miss MPKI, η καλύτερη οργάνωση είναι το BTB-512-1. Το BTB-512-2 έχει ελαφρώς καλύτερο aggregate MPKI, αλλά χειρότερο αριθμητικό μέσο. Η απόκλιση αυτή υποδηλώνει ότι ευνοείται από τα μεγαλύτερα benchmarks, όχι ότι αποτελεί συνολικά πιο σταθερή επιλογή. Για τα συγκεκριμένα πειρά-

ματα η αύξηση των συνολικών entries αποδίδει περισσότερο από την αύξηση associativity, ένδειξη ότι ο περιορισμός είναι κυρίως capacity και όχι καθαρά conflict πρόβλημα. Επομένως επιλέγουμε το BTB-512-1.

5 5.5 Μελέτη του RAS

5.1 Θεωρία και υλοποίηση

Η Return Address Stack (RAS) προβλέπει targets των return instructions. Ένας απλός BTB δυσκολεύεται στα returns, επειδή η ίδια στατική return instruction μπορεί να επιστρέψει σε διαφορετικά δυναμικά call sites. Η RAS εκμεταλλεύεται το γεγονός ότι οι κλήσεις και οι επιστροφές ακολουθούν συνήθως LIFO συμπεριφορά: σε κάθε call γίνεται push της διεύθυνσης επιστροφής και σε κάθε return γίνεται pop και σύγκριση με το πραγματικό target. Συνεπώς, σε θεωρητικό επίπεδο, το κρίσιμο μέγεθος δεν είναι το πλήθος των στατικών return εντολών αλλά το βάθος και η κανονικότητα της δυναμικής στοίβας κλήσεων.

Η βασική μετρική είναι:

$$\text{RAS miss rate} = \frac{\text{incorrect returns}}{\text{correct returns} + \text{incorrect returns}} \cdot 100\%.$$

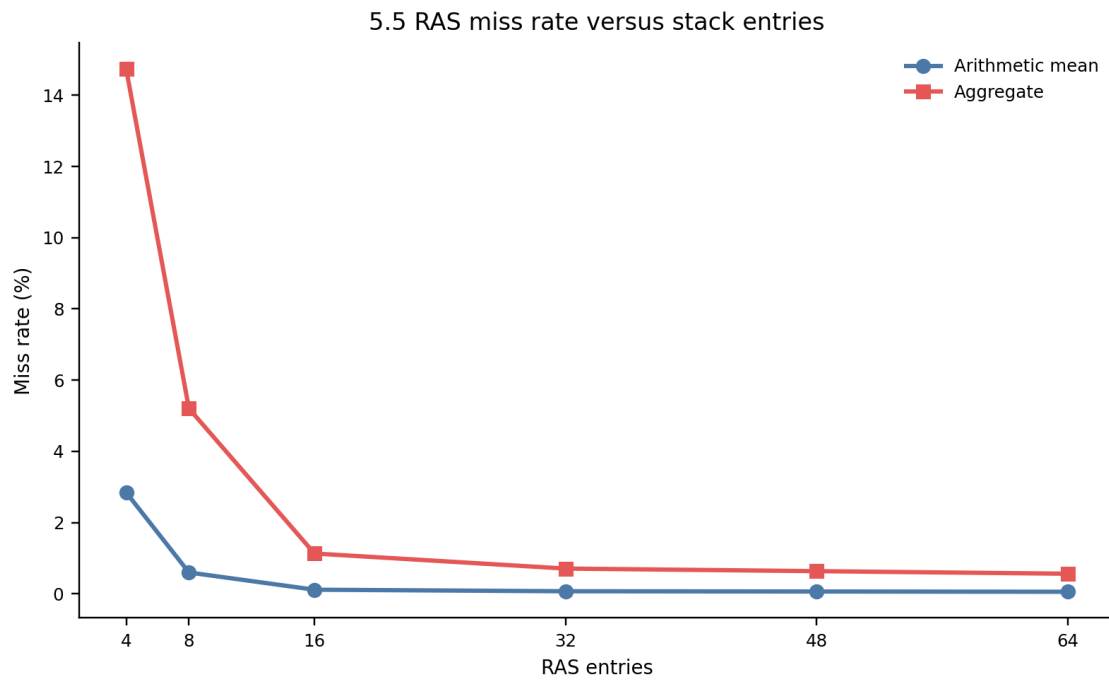
Παράλληλα κρατήσαμε και το RAS miss MPKI, ώστε η επίδραση να είναι κανονικοποιημένη ως προς το μήκος του benchmark.

5.2 Αποτελέσματα και επιλογή

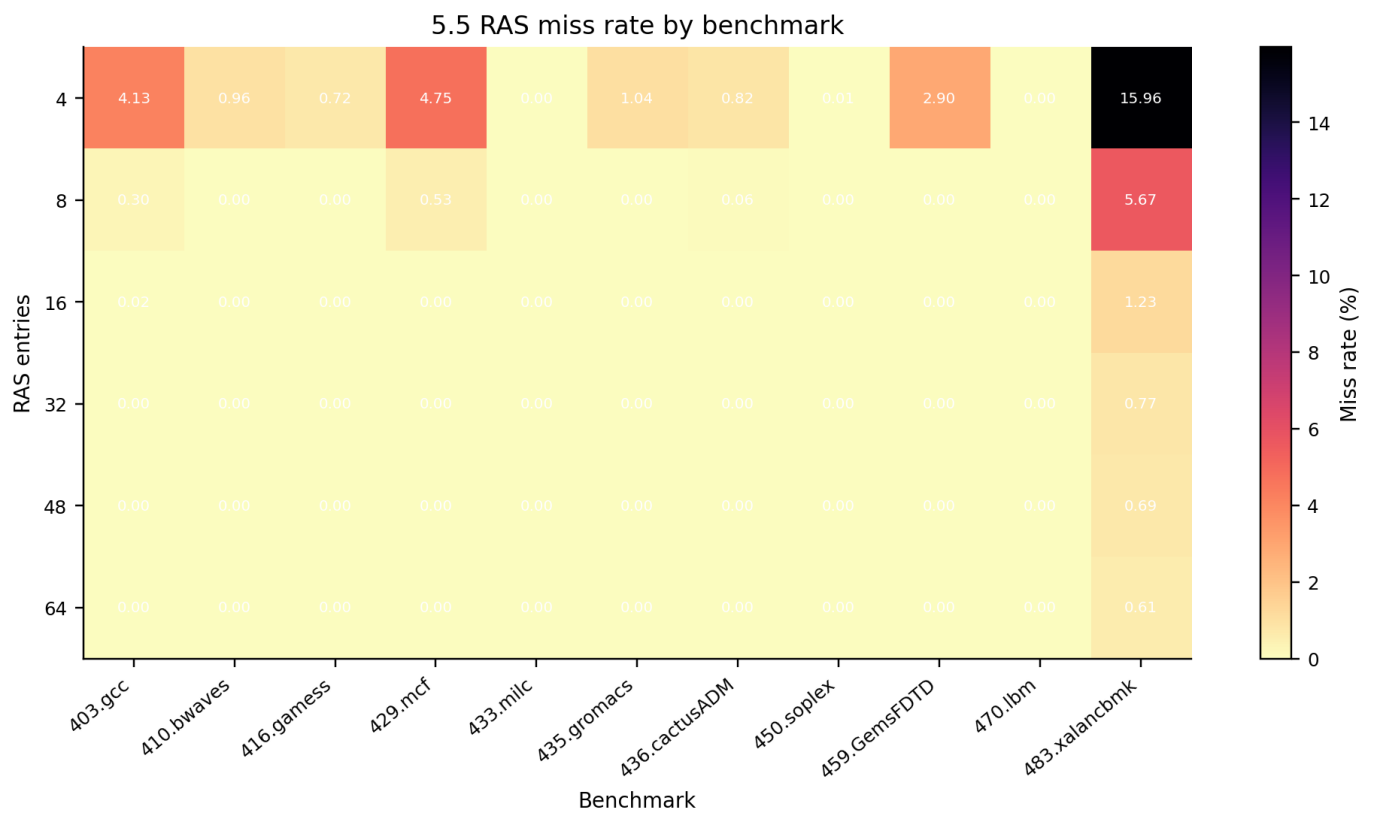
Entries	Mean miss%	Aggregate miss%	Mean MPKI	Aggregate MPKI
4	2.845	14.739	0.777	1.149
8	0.597	5.203	0.254	0.405
16	0.114	1.130	0.054	0.088
32	0.070	0.707	0.034	0.055
48	0.063	0.635	0.030	0.049
64	0.056	0.562	0.027	0.044

Πίνακας 4: Συνοπτικά αποτελέσματα RAS.

Η αύξηση από 4 σε 16 entries μειώνει δραστικά τις αστοχίες. Μετά τα 32 entries η καμπύλη σχεδόν κορέννεται: από 32 σε 64 entries ο αριθμητικός μέσος του miss rate



Σχήμα 7: Ποσοστό αστοχίας RAS για διαφορετικά μεγέθη.



Σχήμα 8: Το RAS miss MPKI ανά benchmark και μέγεθος RAS.

μειώνεται μόνο από 0.070% σε 0.056%. Επομένως, αν το μόνο κριτήριο ήταν η ελάχιστη αστοχία, θα επιλέγαμε 64 entries. Ως αρχιτεκτονική επιλογή όμως προτιμάμε RAS 32 entries, επειδή βρίσκεται πρακτικά στο γόνατο της καμπύλης και δίνει πολύ μικρή τιμή του miss MPKI με μισό μέγεθος σε σχέση με τα 64 entries. Η επιλογή αυτή συνδέεται με τη θεωρία της RAS: μόλις καλυφθεί το τυπικό δυναμικό βάθος κλήσεων, η περαιτέρω αύξηση του stack προσφέρει μόνο οριακή βελτίωση.

6 5.6.1 Perceptrons

6.1 Θεωρητικό υπόβαθρο

Ο μηχανισμός πρόβλεψης perceptron χρησιμοποιεί global history length n και έναν πίνακα από M perceptrons. Η διεύθυνση του branch επιλέγει ένα perceptron. Κάθε perceptron έχει $n + 1$ βάρη:

$$w_0, w_1, \dots, w_n,$$

όπου το w_0 είναι bias. Τα τελευταία n branch outcomes κωδικοποιούνται ως:

$$x_i = \begin{cases} -1, & \text{αν το branch ήταν not-taken,} \\ 1, & \text{αν το branch ήταν taken.} \end{cases}$$

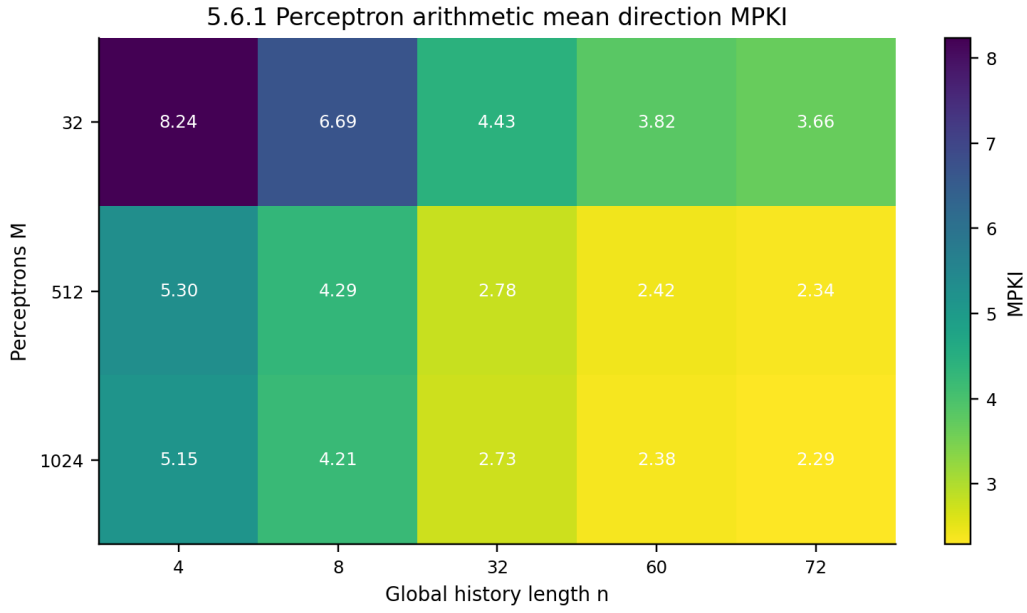
Η έξοδος είναι:

$$y = w_0 + \sum_{i=1}^n x_i w_i.$$

Αν $y \geq 0$, η πρόβλεψη είναι taken, αλλιώς not-taken. Η εκπαίδευση γίνεται όταν η πρόβλεψη είναι λάθος ή όταν είναι σωστή αλλά με χαμηλή εμπιστοσύνη:

$$|y| \leq \theta, \quad \theta = \lfloor 1.93n + 14 \rfloor.$$

Στην υλοποίησή μας κρατήσαμε την ονοματολογία Perceptron-M<M>-N<n> και περιορίσαμε τα βάρη στο διάστημα $[-\theta, \theta]$, ώστε ο υπολογισμός του hardware κόστους στο επόμενο ερώτημα να είναι συνεπής με το πλήθος bits ανά βάρος. Σε σχέση με τους saturating counters, το perceptron λειτουργεί ως γραμμικός ταξινομητής πάνω στο πρόσφατο global history. Επομένως, μπορεί να αξιοποιήσει μακρύτερες συσχετίσεις μεταξύ διαφορετικών branches, με αντάλλαγμα μεγαλύτερο κόστος αποθήκευσης βαρών και τον υπολογισμό του dot product σε κάθε πρόβλεψη.



Σχήμα 9: Αριθμητικός μέσος του direction MPKI για τα απαιτούμενα ζεύγη M, n .

6.2 Σχολιασμός αποτελεσμάτων

Το Σχήμα 9 δείχνει ότι το global history length n είναι ο κυρίαρχος παράγοντας. Η μέση τιμή του MPKI ανά n , συνυπολογίζοντας τα τρία M , μειώνεται από 6.229 για $n = 4$ σε 2.763 για $n = 72$. Η αύξηση του M βοηθά επίσης, επειδή μειώνει το aliasing μεταξύ διαφορετικών branch PCs, αλλά το όφελος από 512 σε 1024 perceptrons είναι μικρότερο από το όφελος αύξησης του n . Αυτό συμφωνεί με τη θεωρία των perceptrons: όταν υπάρχει ουσιαστικό global correlation, το μεγαλύτερο history length προσθέτει πληροφορία που δεν μπορεί να αντικατασταθεί απλώς από περισσότερες entries.

Predictor	M	n	Mean MPKI	Aggregate MPKI
Perceptron-M1024-N72	1024	72	2.287	1.333
Perceptron-M512-N72	512	72	2.336	1.339
Perceptron-M1024-N60	1024	60	2.375	1.472
Perceptron-M512-N60	512	60	2.424	1.477
Perceptron-M1024-N32	1024	32	2.732	2.025

Πίνακας 5: Καλύτερες διαμορφώσεις perceptron στο 5.6.1.

Η καλύτερη διαμόρφωση του ερωτήματος είναι η Perceptron-M1024-N72, με τιμή mean MPKI 2.287 και τιμή aggregate MPKI 1.333. Το αποτέλεσμα αυτό δείχνει την αξία του μεγάλου global history length για branches που συσχετίζονται με παλαιότερα outcomes.

Ωστόσο, η διαμόρφωση αυτή δεν είναι συγκρίσιμη με το 32K-bit budget του 5.6.2, επειδή απαιτεί πολύ μεγαλύτερο πίνακα βαρών. Για τον λόγο αυτό στο επόμενο ερώτημα επιλέγουμε μικρότερες διαμορφώσεις perceptron περίπου 32K bits.

7 5.6.2 Σύγκριση διαφορετικών μηχανισμών πρόβλεψης

7.1 Υλοποίηση και επιλογές hardware

Στο τελικό συγκριτικό πείραμα αξιολογήσαμε 18 predictors. Εκτός από τους ήδη διαθέσιμους μηχανισμούς, υλοποιήθηκαν static predictors, two-level predictors με local history, two-level predictors με global history, ένας Alpha 21264-style hybrid predictor και τέσσερις tournament predictors. Η επιλογή αυτή καλύπτει τις βασικές ιδέες της θεωρίας: στατική πρόβλεψη, απλή δυναμική πρόβλεψη με counters, αξιοποίηση local history ή global history και υβριδική επιλογή ανάμεσα σε επιμέρους predictors.

Για τους local-history predictors κρατήσαμε σταθερό:

$$\text{PHT} = 8192 \cdot 2 = 16K \text{ bits.}$$

Το υπόλοιπο budget δίνεται στο BHT, άρα για συνολικό κόστος 32K bits:

$$(X, Z) = (2048, 8), (4096, 4), (8192, 2).$$

Για τους global-history predictors αγνοήσαμε το μικρό κόστος του BHR, όπως επιτρέπει η εκφώνηση. Επομένως:

$$Z \cdot 2 = 32K \Rightarrow Z = 16K \text{ PHT entries,}$$

και συγκρίνουμε BHR μήκη 4, 8 και 12.

Για τα perceptrons χρησιμοποιήσαμε την προσέγγιση:

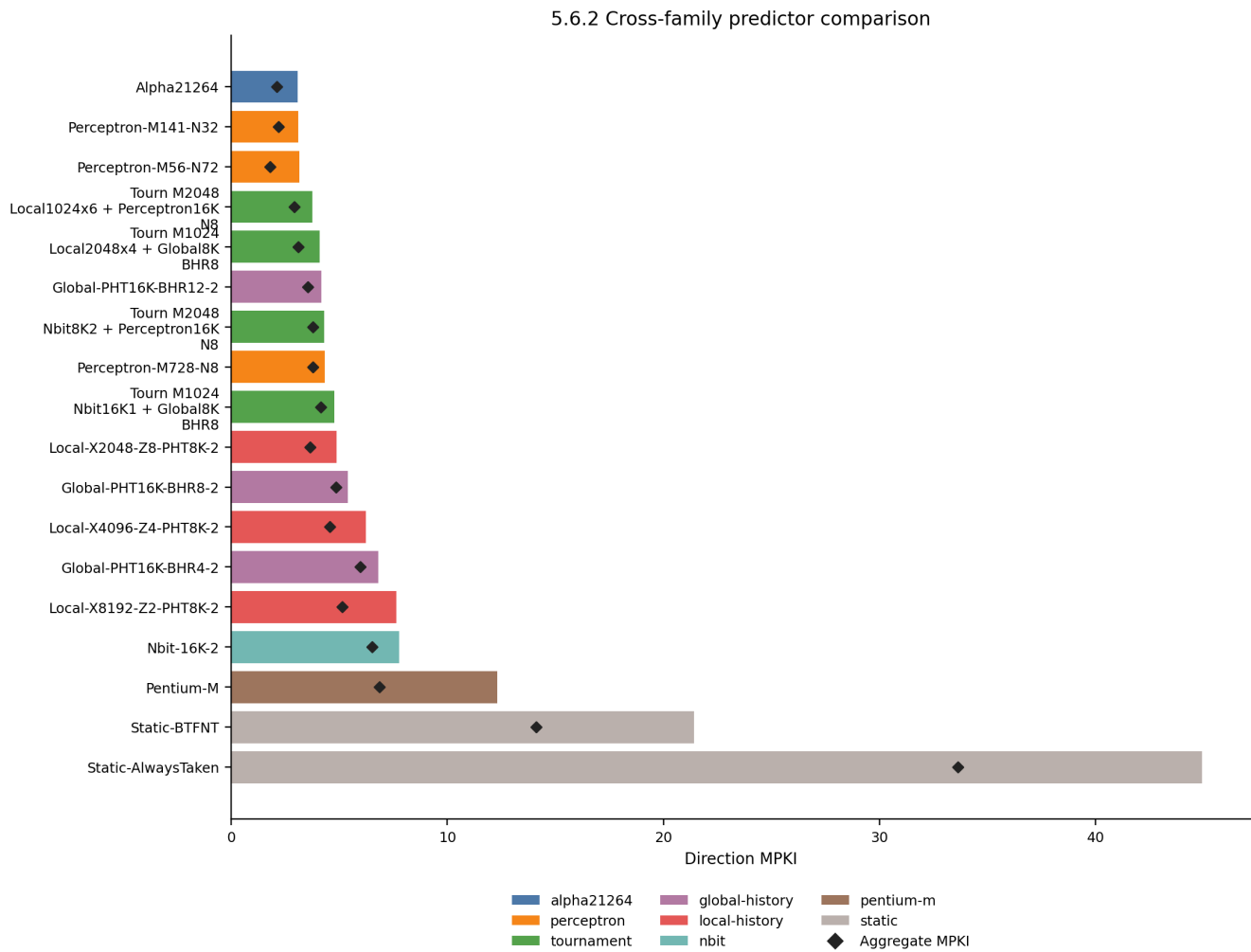
$$\text{bits} = M(n+1)(1 + \lfloor \log_2 \theta \rfloor), \quad \theta = \lfloor 1.93n + 14 \rfloor.$$

Οι τρεις επιλογές κοντά στα 32K bits είναι:

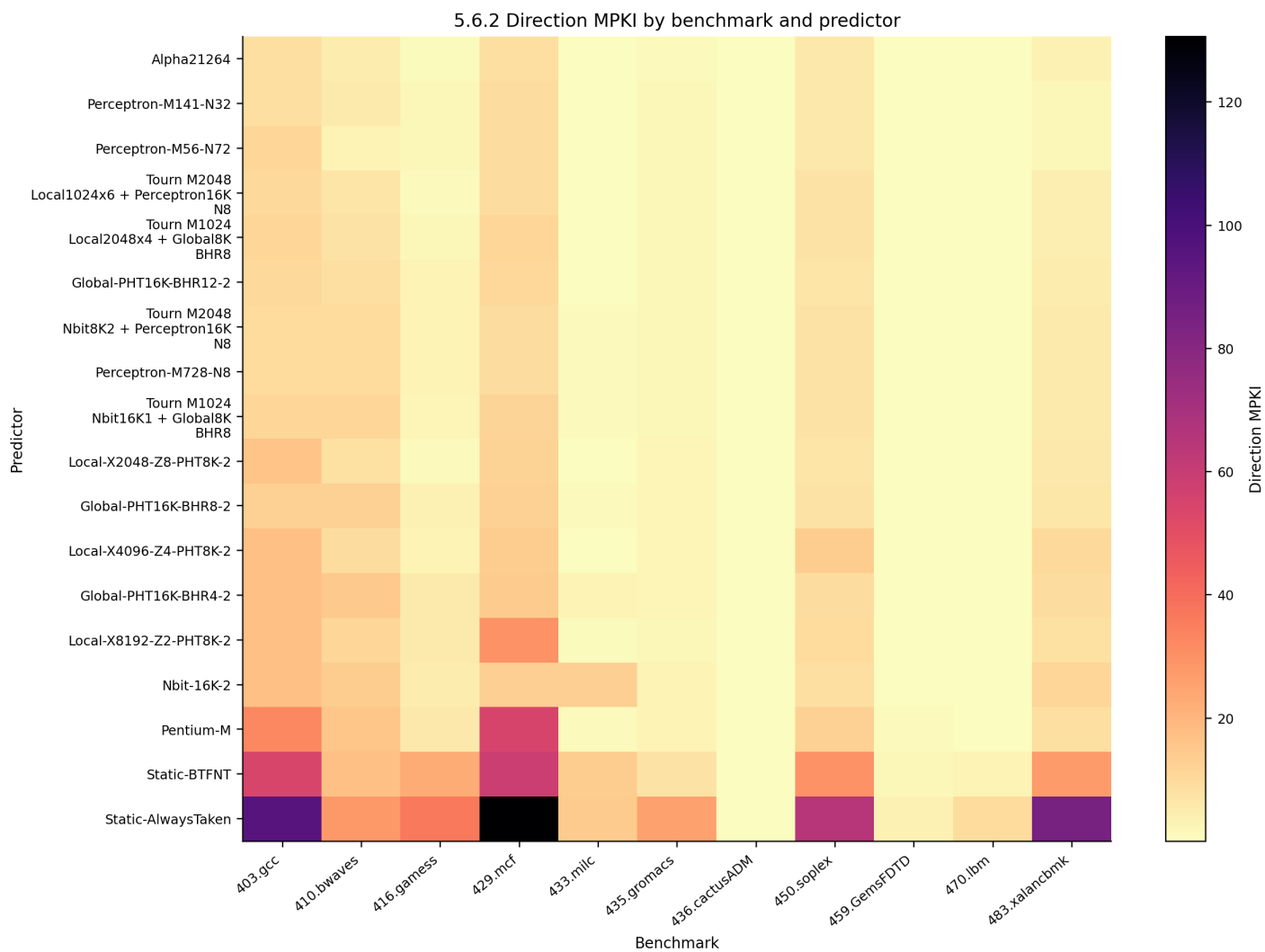
$$\text{M728-N8} : 32760 \text{ bits,} \quad \text{M141-N32} : 32571 \text{ bits,} \quad \text{M56-N72} : 32704 \text{ bits.}$$

Ο Alpha21264 υλοποιήθηκε με local history table 1024 entries των 10 bits, local PHT 1024 entries των 3 bits, global PHT 4096 entries των 2 bits και choice PHT 4096 entries των 2 bits, δηλαδή συνολικά 29696 bits. Στους tournament predictors αγνοήσαμε το κόστος του meta-predictor, όπως ζητείται, και δώσαμε περίπου 16K bits σε κάθε συνιστώσα

P_0 και P_1 . Ο ρόλος του choice/meta predictor είναι κρίσιμος θεωρητικά: δεν προσπαθεί να προβλέψει απευθείας το branch, αλλά μαθαίνει ποιος επιμέρους predictor είναι πιο αξιόπιστος για το τρέχον pattern.



Σχήμα 10: Σύγκριση των 18 predictors του 5.6.2. Οι μπάρες δείχνουν τον αριθμητικό μέσο του MPKI και τα μαύρα σημεία δείχνουν το aggregate MPKI.



Σχήμα 11: To direction MPKI ανά benchmark για τους predictors του 5.6.2.

7.2 Αποτελέσματα και τελική επιλογή

Predictor	Family	Bits	Mean MPKI	Aggregate MPKI
Alpha21264	alpha21264	29696	3.071	2.119
Perceptron-M141-N32	perceptron	32571	3.088	2.186
Perceptron-M56-N72	perceptron	32704	3.140	1.788
Tournament-M2048-Local1024x6-Perceptron16K-N8	tournament	30716	3.743	2.922
Tournament-M1024-Local2048x4-Global18K-BHR8	tournament	32768	4.091	3.102
Global-PHT16K-BHR12-2	global-history	32768	4.181	3.555
Perceptron-M728-N8	perceptron	32760	4.314	3.768
Nbit-16K-2	nbit	32768	7.770	6.505
Static-BTFNT	static	0	21.430	14.108
Static-AlwaysTaken	static	0	44.920	33.623

Πίνακας 6: Επιλεγμένες γραμμές από τη συνολική σύγκριση των 18 predictors.

Η ιεράρχηση είναι σαφής. Οι static predictors είναι χρήσιμοι ως baselines, αλλά απέχουν πολύ από τους dynamic predictors, επειδή δεν προσαρμόζονται στη συμπεριφορά κάθε στατικού branch. Ο κανόνας Static-BTFNT είναι καλύτερος από τον Static-AlwaysTaken, επειδή εκμεταλλεύεται τη συχνή συμπεριφορά των backward loop branches, αλλά παραμένει απλός στατικός κανόνας.

Οι local-history και global-history predictors βελτιώνουν σημαντικά τον 2-bit μηχανισμό. Στην κατηγορία των global-history predictors, η καλύτερη διαμόρφωση είναι η Global-PHT16K-BHR12-2, με 4.181 mean MPKI. Η μέτρηση αυτή δείχνει ότι το global correlation είναι σημαντικό στα συγκεκριμένα benchmarks. Οι local-history predictors είναι ιδιαίτερα χρήσιμοι όταν ένα συγκεκριμένο static branch ακολουθεί δικό του επαναλαμβανόμενο μοτίβο, ενώ οι global-history predictors μπορούν να εκμεταλλευθούν συσχετίσεις μεταξύ διαφορετικών branches. Στα δεδομένα μας οι global-history predictors αποδίδουν καλύτερα, γεγονός που υποδηλώνει ότι υπάρχει αξιοποιήσιμο inter-branch correlation. Οι tournament predictors βελτιώνουν περαιτέρω τις συνιστώσες τους, ειδικά όταν μία από αυτές είναι perceptron predictor. Ωστόσο, η καλύτερη διαμόρφωση για tournament predictors επιτυγχάνει 3.743 mean MPKI και δεν φτάνει τους κορυφαίους predictors.

Οι τρεις καλύτερες διαμορφώσεις με βάση τον αριθμητικό μέσο είναι η Alpha21264, η Perceptron-M141-N32 και η Perceptron-M56-N72. Η Alpha21264 είναι πρώτη σε mean MPKI, με 3.071, και έχει μικρότερο κόστος από τις δύο διαμορφώσεις perceptron. Η

Perceptron-M56-N72 έχει την καλύτερη τιμή aggregate MPKI, 1.788, επειδή αποδίδει πολύ καλά στα μεγάλα benchmarks, αλλά έχει ελαφρώς χειρότερο αριθμητικό μέσο. Ως τελική επιλογή μετά το 5.6.2 θα προκρινόταν η Alpha21264: συνδυάζει local history και global history, διαθέτει choice predictor που μαθαίνει ποια συνιστώσα είναι καλύτερη, έχει την καλύτερη ίσου-βάρους επίδοση και δεν απαιτεί dot product πολλών βαρών σε κάθε πρόβλεψη.

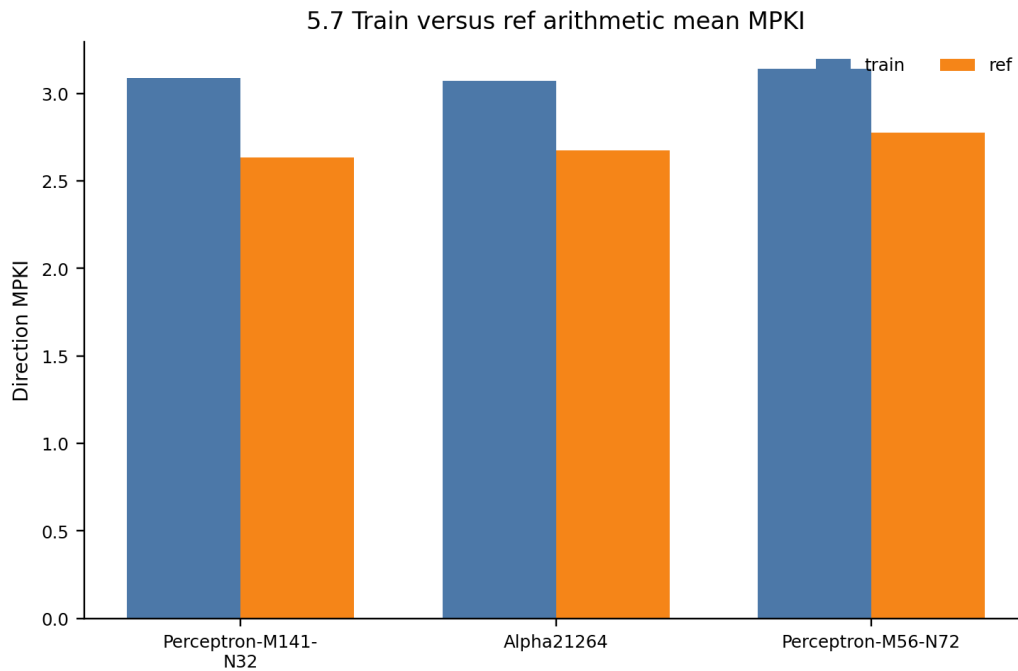
8 5.7 Ακρίβεια προσομοιώσεων με ref inputs

8.1 Επιλογή μηχανισμών πρόβλεψης για έλεγχο

Το 5.7 ελέγχει αν τα συμπεράσματα που προέκυψαν από τα train inputs παραμένουν αξιόπιστα στα μεγαλύτερα ref inputs. Επιλέξαμε τους τρεις καλύτερους μηχανισμούς του 5.6.2 με βάση τον αριθμητικό μέσο του direction MPKI:

Alpha21264, Perceptron-M141-N32, Perceptron-M56-N72.

Δεν επιλέξαμε τις πολύ μεγάλες διαμορφώσεις perceptron του 5.6.1, παρότι έχουν χαμηλότερο MPKI, επειδή ξεπερνούν κατά πολύ το περίπου 32K-bit budget και άρα δεν θα ήταν δίκαιη σύγκριση.



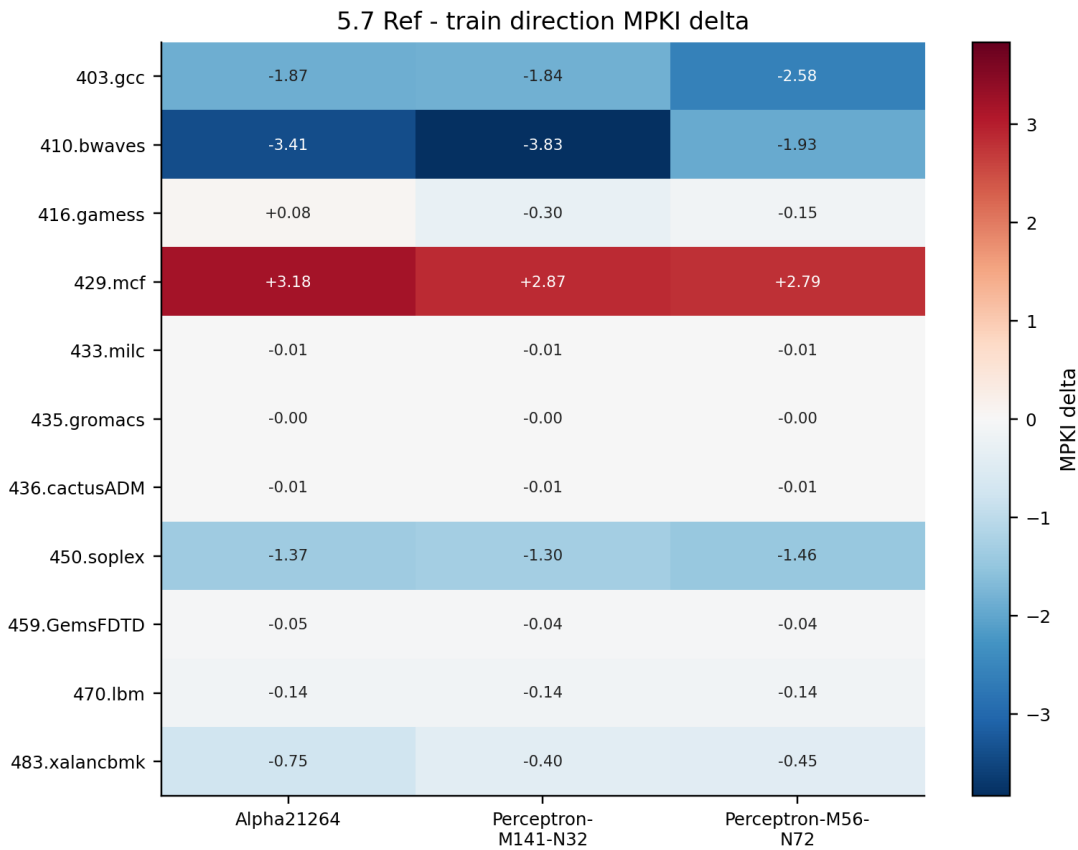
Σχήμα 12: Αριθμητικός μέσος του direction MPKI σε train και ref inputs για τους τρεις επιλεγμένους μηχανισμούς.

Predictor	Train mean	Ref mean	Train agg.	Ref agg.
Alpha21264	3.071	2.675	2.119	1.129
Perceptron-M141-N32	3.088	2.634	2.186	1.139
Perceptron-M56-N72	3.140	2.778	1.788	1.110

Πίνακας 7: Σύγκριση train και ref για τους τρεις επιλεγμένους μηχανισμούς.

8.2 Σύγκριση συμπερασμάτων

Στα ref inputs και οι τρεις μηχανισμοί βελτιώνουν τον αριθμητικό μέσο του MPKI σε σχέση με τα train inputs. Η σειρά όμως αλλάζει οριακά: η Perceptron-M141-N32 γίνεται πρώτη με 2.634 MPKI, η Alpha21264 ακολουθεί με 2.675 MPKI και η Perceptron-M56-N72 έχει 2.778 MPKI. Η διαφορά των δύο πρώτων είναι περίπου 0.04 MPKI, δηλαδή πολύ μικρή για να θεωρηθεί ανατροπή του συμπεράσματος. Η μικρή αυτή διαφορά υποδηλώνει ότι τα train inputs ήταν αρκετά αντιπροσωπευτικά για την κατάταξη της κορυφαίας ομάδας, παρότι δεν προβλέπουν τέλεια κάθε επιμέρους benchmark.



Σχήμα 13: Μεταβολή του direction MPKI ανά benchmark από train σε ref. Αρνητική τιμή σημαίνει καλύτερη επίδοση στο ref input.

Το Σχήμα 13 δείχνει ότι η μεγαλύτερη επιδείνωση εμφανίζεται στο 429.mcf, όπου και οι τρεις μηχανισμοί έχουν υψηλότερο MPKI στο ref. Αντίθετα, στα 403.gcc, 410.bwaves και 450.soplex η επίδοση βελτιώνεται. Το συμπέρασμα από το train παραμένει ουσιαστικά σταθερό: οι τρεις επιλεγμένοι μηχανισμοί αποτελούν την κορυφαία ομάδα, αλλά οι μεταξύ τους διαφορές είναι μικρές και εξαρτώνται από το αν χρησιμοποιούμε αριθμητικό ή aggregate μέσο.

Σε aggregate MPKI η σειρά στα ref είναι Perceptron-M56-N72, Alpha21264, Perceptron-M141-N32, με τιμές 1.110, 1.129 και 1.139 αντίστοιχα. Αυτό επιβεβαιώνει ότι η διαμόρφωση perceptron με μεγάλο global history length είναι πολύ ισχυρή στα benchmarks που έχουν μεγάλο βάρος σε δυναμικές εντολές. Ωστόσο, η Alpha21264 παραμένει η πιο ισορροπημένη τελική επιλογή: ήταν πρώτη στο train mean, είναι πρακτικά ισοδύναμη με την πρώτη στο ref mean, έχει καλύτερο ref aggregate από την Perceptron-M141-N32, χαμηλότερο κόστος και μικρότερη υπολογιστική πολυπλοκότητα ανά πρόβλεψη.

9 Συνολικά συμπεράσματα

Η άσκηση καταδεικνύει ότι η πρόβλεψη διακλαδώσεων δεν είναι μονοδιάστατο πρόβλημα. Το 5.2 έδειξε ότι τα benchmarks έχουν πολύ διαφορετικό branch frequency και διαφορετικό μίγμα conditional branches, calls και returns. Αυτό εξηγεί γιατί απαιτούνται ξεχωριστοί μηχανισμοί για direction prediction, target prediction και return prediction.

Στους απλούς μηχανισμούς n-bit πρόβλεψης, ο 2-bit saturating counter παραμένει η καλύτερη πρακτική επιλογή όταν το hardware budget είναι σταθερό στα 32K bits. Τα Nair FSMs επιβεβαιώνουν ότι υπάρχουν σχεδόν ισοδύναμες 4-state μηχανές, αλλά δεν δίνουν ουσιαστικό κέρδος έναντι της κλασικής και απλούστερης μορφής. Στο BTB, η καλύτερη οργάνωση με βάση τον αριθμητικό μέσο είναι το BTB-512-1, επειδή το πρόσθετο πλήθος entries αποδίδει περισσότερο από το πρόσθετο associativity. Στη RAS, η επιλογή 32 entries προσφέρει σχεδόν όλη την ωφέλεια των μεγαλύτερων stacks με μικρότερο κόστος.

Τα perceptrons εκμεταλλεύονται αποτελεσματικά μεγάλο global history length και πετυχαίνουν πολύ χαμηλό MPKI όταν δεν υπάρχει αυστηρός περιορισμός hardware. Στο πλαίσιο δίκαιης σύγκρισης με budget 32K bits, η Alpha21264 είναι ο καλύτερος συνολικός συμβιβασμός. Συνδυάζει local history και global history, χρησιμοποιεί choice predictor, έχει κόστος περίπου 29K bits και παραμένει στην κορυφαία ομάδα και στα ref inputs. Επομένως, αν έπρεπε να υλοποιηθεί ένας μηχανισμός πρόβλεψης στο σύστημα, η τελική

επιλογή θα ήταν η Alpha21264.